

ml-lab-ass-5

July 18, 2025

1 Assignment 5

1.1 Step 1: Import Required Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.decomposition import PCA
```

1.2 Step 2: Load and Explore the Dataset

```
[2]: df = pd.read_csv('sales_data_sample.csv', encoding='ANSI') # Make sure it's in
    ↳ the same folder
df.head()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2823 entries, 0 to 2822
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ORDERNUMBER           2823 non-null  int64
1   QUANTITYORDERED       2823 non-null  int64
2   PRICEEACH             2823 non-null  float64
3   ORDERLINENUMBER       2823 non-null  int64
4   SALES                 2823 non-null  float64
5   ORDERDATE             2823 non-null  object
6   STATUS                2823 non-null  object
7   QTR_ID               2823 non-null  int64
8   MONTH_ID             2823 non-null  int64
9   YEAR_ID              2823 non-null  int64
10  PRODUCTLINE           2823 non-null  object
11  MSRP                 2823 non-null  int64
```

```

12 PRODUCTCODE      2823 non-null  object
13 CUSTOMERNAME     2823 non-null  object
14 PHONE            2823 non-null  object
15 ADDRESSLINE1     2823 non-null  object
16 ADDRESSLINE2     302 non-null   object
17 CITY             2823 non-null  object
18 STATE            1337 non-null  object
19 POSTALCODE       2747 non-null  object
20 COUNTRY          2823 non-null  object
21 TERRITORY        1749 non-null  object
22 CONTACTLASTNAME  2823 non-null  object
23 CONTACTFIRSTNAME 2823 non-null  object
24 DEALSIZE         2823 non-null  object
dtypes: float64(2), int64(7), object(16)
memory usage: 551.5+ KB

```

1.3 Step 3: Data Preprocessing

```

[3]: # Drop non-numeric columns (keep only numeric ones for clustering)
df_numeric = df.select_dtypes(include=[np.number])

# Optional: drop columns like 'PHONE', 'POSTALCODE' if needed
df_numeric = df_numeric.dropna() # Remove rows with NaN values

# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_numeric)

```

1.4 Step 4: Elbow Method to Find Optimal Clusters (K)

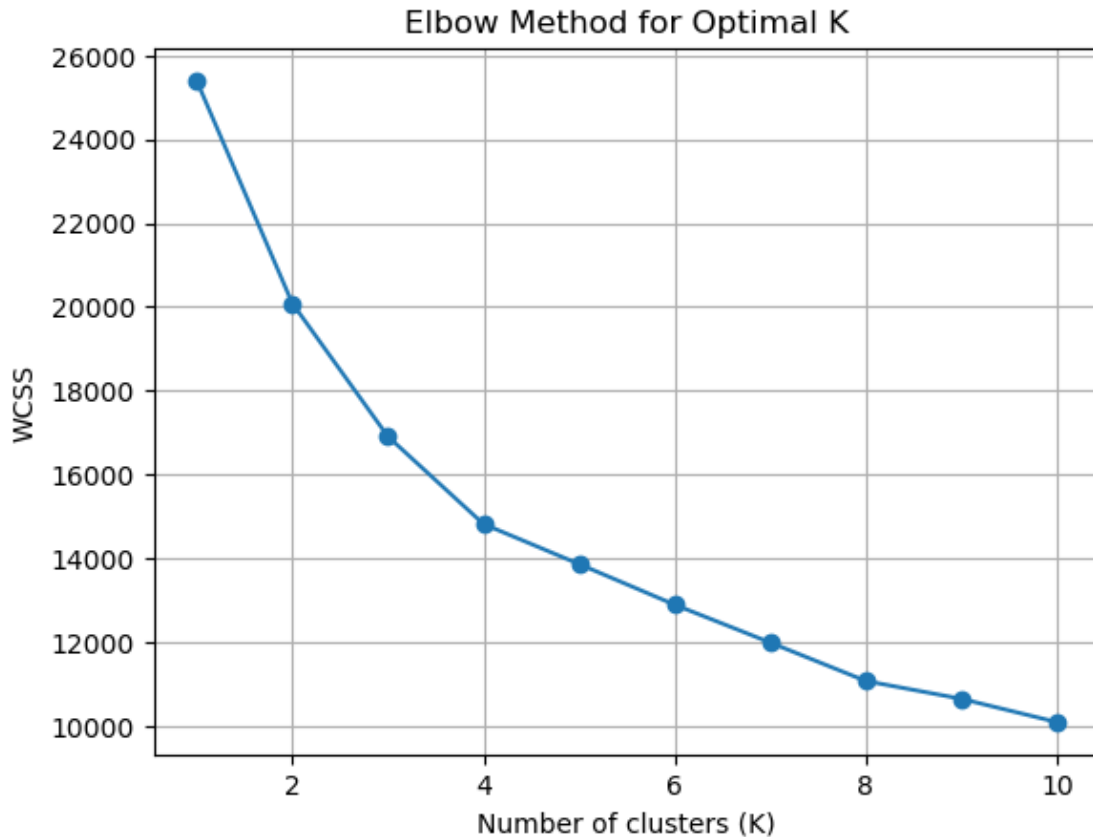
```

[4]: wcss = []

for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42)
    kmeans.fit(scaled_data)
    wcss.append(kmeans.inertia_)

# Plotting the Elbow graph
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()

```



1.5 Observations

1. The WCSS drops sharply from K=1 to K=3 or 4.
2. After K=4, the curve starts to flatten — meaning additional clusters don't reduce WCSS significantly. ## Conclusion
3. The optimal number of clusters (K) is likely 4.
4. This is where the “elbow” appears — the point after which adding more clusters yields diminishing returns in reducing WCSS.

1.6 Step 5: Apply K-Means Clustering

```
[5]: optimal_k = 4 # Replace with the value from elbow method
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42)
clusters = kmeans.fit_predict(scaled_data)

# Add cluster labels to original data
df['Cluster_KMeans'] = clusters

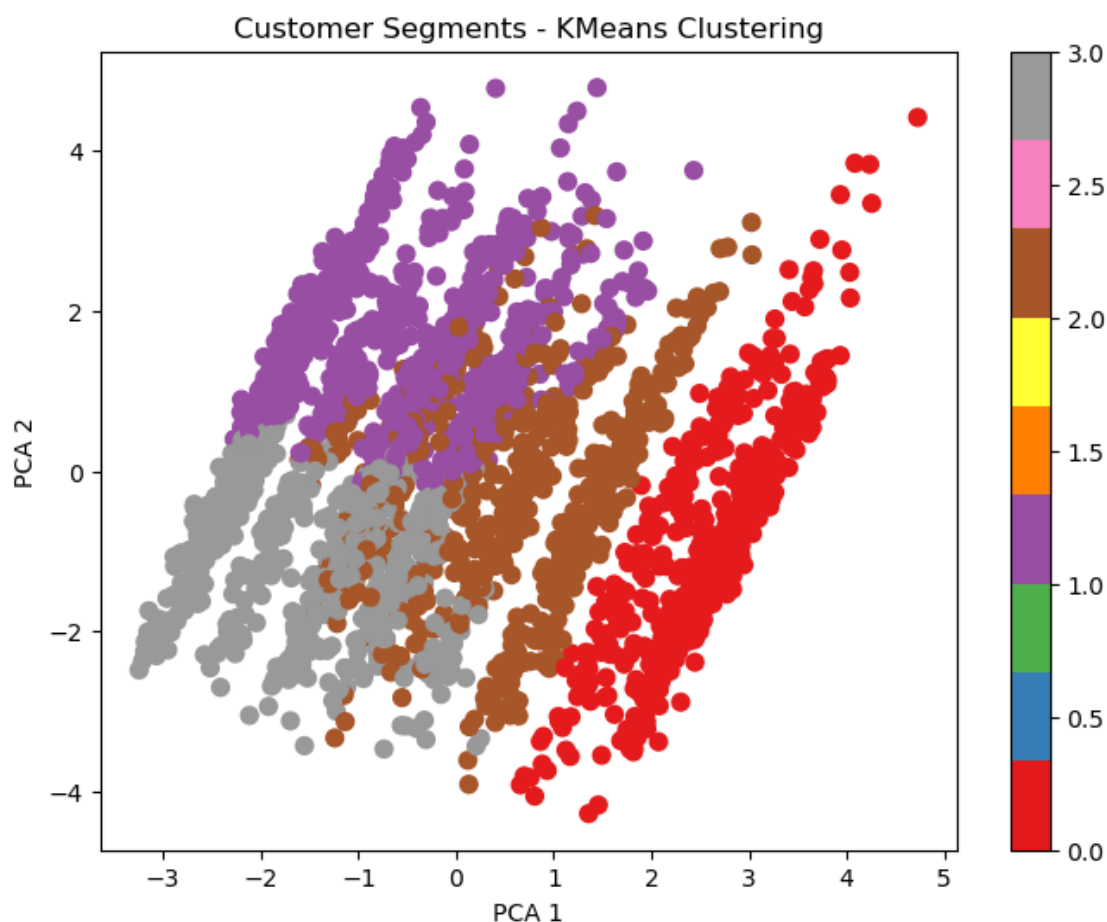
# Check few entries
df[['Cluster_KMeans']].value_counts()
```

```
[5]: Cluster_KMeans
      1          948
      3          734
      2          663
      0          478
      Name: count, dtype: int64
```

1.7 Step 6: Visualize Clusters (Optional using PCA)

```
[6]: # Reduce dimensions for plotting
      pca = PCA(n_components=2)
      pca_data = pca.fit_transform(scaled_data)

      plt.figure(figsize=(8,6))
      plt.scatter(pca_data[:,0], pca_data[:,1], c=clusters, cmap='Set1', s=50)
      plt.title('Customer Segments - KMeans Clustering')
      plt.xlabel('PCA 1')
      plt.ylabel('PCA 2')
      plt.colorbar()
      plt.show()
```



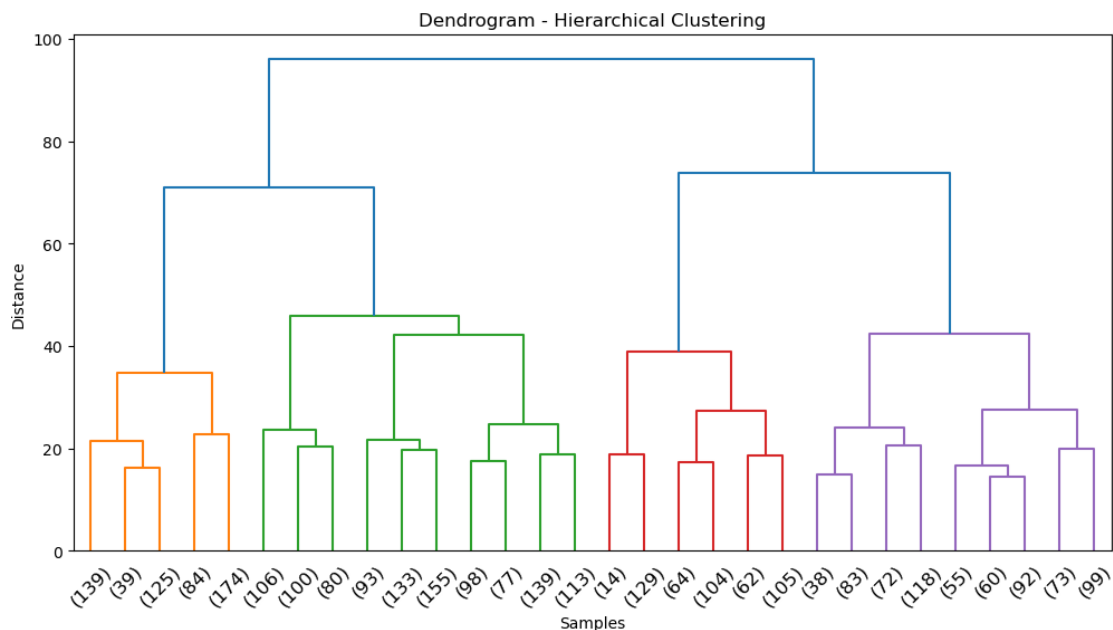
1.8 Conclusion

Using PCA for dimensionality reduction, the K-Means clustering clearly separated the data into 4 distinct customer segments. Each cluster demonstrates unique characteristics in the dataset, which is valuable for customer profiling or targeted marketing strategies.

1.9 Step 7: Hierarchical Clustering

```
[7]: linked = linkage(scaled_data, method='ward') # or 'complete', 'average'

plt.figure(figsize=(12, 6))
dendrogram(linked, truncate_mode='lastp', p=30, leaf_rotation=45,
            leaf_font_size=12)
plt.title('Dendrogram - Hierarchical Clustering')
plt.xlabel('Samples')
plt.ylabel('Distance')
plt.show()
```



1.10 Conclusion

The dendrogram shows clear hierarchical relationships between samples. By analyzing the vertical distances between merges, we conclude that the data naturally forms approximately 4 clusters. This validates the result we got from the elbow method in K-Means. Hierarchical clustering is

helpful for visualizing nested subgroupings within customer data without predefining the number of clusters.